

## Vectorization (Part 1)

When implementing a linear algorithm, using vectorization will make both your code shorter & more efficiently.

- Concrete example of what vectorization means:

$$\vec{w} = [w_1, w_2, w_3] \quad n=3$$

$b$  is a number

$$\vec{x} = [x_1, x_2, x_3]$$

In linear Algebra  
value counts from  
1.

→ In python code, you can define these variable  $w$ ,  $b$  &  $x$  using arrays like: using "numpy".

$$w = \text{np.array}([1.0, 2.5, -3.3])$$

$$b = 4$$

$$x = \text{np.array}([10, 20, 30])$$

$$x[0], x[1], x[2]$$

here indexing is  
from 0. as  
you know in  
coding.

- Implementation w/o vectorization:

$$\hat{f}_{wb}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

In code it will look like:



1). •  $f = w[0] * x[0] + w[1] * x[1] + w[2] * x[2] + b$

but what if  $n = 100000$ , writing like this would be very inefficient for you & your programming / computer.

In Math:

You can write as,

$$f_{\vec{w}, b}(\vec{x}) = \left( \sum_{j=1}^n w_j x_j \right) + b$$



2). in code:



1).  $f = 0$   $j = 0 \text{ to } n-1$

2). for  $j$  in range  $(0, n)$ :

3).  $f = f + w[j] * x[j]$

4).  $f = f + b$

↳ this is okayish / bit better (doesn't use vectorization, & isn't that efficient).



3). Using vectorization:

$$f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Now we can implement this by single line of code:

$f = \text{np.dot}(w, x) + b$

this numpy function is the dot product operation b/w two vector & especially when  $n$  is large this will run much faster.

this dot function is able to use parallel hardware in your computer (normal or GPU)